# Advanced Memory Allocation

Contiguous and Superpage-Aware Memory Allocation

| | |
|---|---|
| Course Name | : Operating Systems Lab |
| Course Code | : CSE 4502 |
| Course Instructor | : S. M. Sabit Bananee<br>Dr. Muhammad Mahbub Alam |
| Group | : A(odd) |
| Team No. | : 3 |
| Project No. | : 10 |
| Project Name | : Advanced Memory Allocation |

# Index

# mCertikOS

The memory allocation in mCertiKOS is primitive. It only allows the user to allocate one 4K page upon a page fault. In many cases, allocating pages that are consecutive in both virtual and physical addresses is useful, especially for applications that require operations with the DMA engine. Another benefit of allocating multiple consecutive pages at a time is the ability to combine them into a super-page. A super-page (e.g., Page Size Extension bit in x86) allows the MMU to reduce TLB entries and further reduce page table switch overhead.

Limitation of base system

- No structured free memory tracking
- No efficient handling of fragmentation
- No support for large memory requests
- Inefficient due to high TLB misses and page table overhead

So in this project, we implemented an improved page allocation system that allows the user to:

- Allocate physically consecutive pages
- Allocate super pages
- Free pages

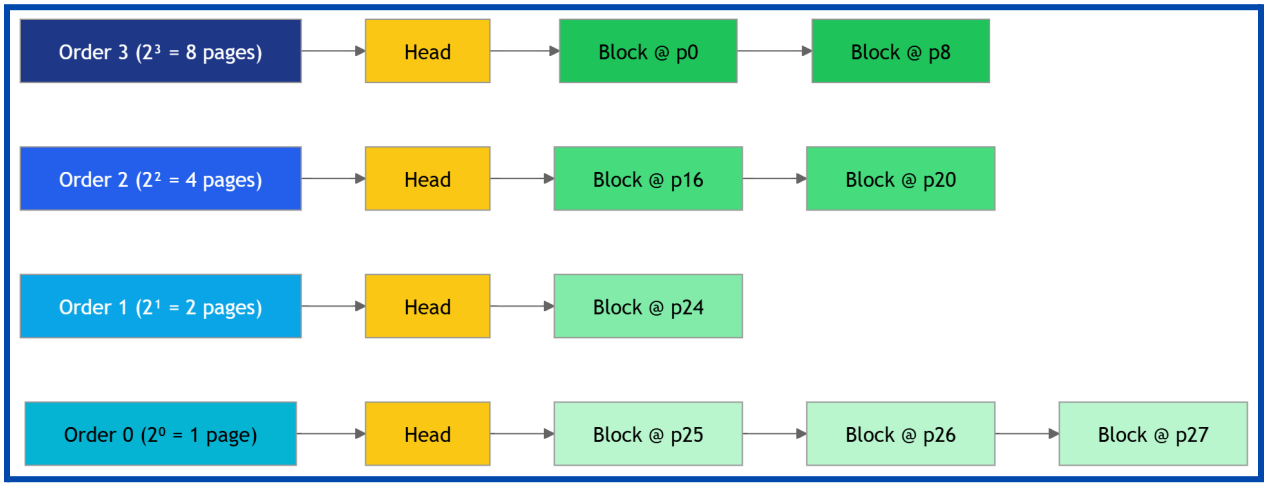
with syscall brk(). Besides, the page allocation system should be robust and could handle page fragmentation efficiently.

# Added/Modified Features

We extended the base system by implementing the following major modules:

### Free List Architecture

The free list structure organizes memory into multiple linked lists, where each list corresponds to blocks of size (2^order). Each node represents the starting address of a free block.. This structure allows constant-time insertion/removal and supports the buddy system's splitting and merging operations. This Enables O(1) insertion and deletion.




### Buddy Allocation System

This implements power-of-two memory allocation (2^order) and uses a free list data structure for each order. This supports:

- Block splitting
- Block merging

### Allocation Table (AT[])

The Allocation Table (AT[]) is an array that stores metadata for each physical page, including allocation status, permissions, and block order. It enables efficient tracking of memory usage and supports the buddy allocator in managing splitting and merging operations. This structure ensures fast and organized memory management.


### Superpage Allocation & Deallocation

Superpage allocation provides large contiguous memory blocks (4MB) to improve performance by reducing TLB misses and page table overhead. The system first attempts to allocate superpages from a dedicated memory zone and falls back to normal pages if unavailable. During deallocation, the superpage is freed and returned to the buddy allocator, where it may be split or merged with adjacent free blocks.

### brk() System Call Integration

The brk system call is used to control the end of a process's data segment (the heap). It adjusts the program break, allowing the process to request more heap memory or release unused memory. It is one of the basic mechanisms behind dynamic memory allocation in Unix-like systems. It handles :

- Page allocation
- Page deallocation

While implementing, we added extra fields in the Allocation Table in **MATIntro** layer

```
STRUCT ATStruct
    perm        : UNSIGNED INTEGER
    allocated   : UNSIGNED INTEGER
    order       : UNSIGNED INTEGER
    next        : INTEGER
    prev        : INTEGER
END STRUCT
```

Then in the **MATInit** layer, we modified the pmem_init() with buddy allocation strategy.

```
FUNCTION pmem_init(mbi_addr)
    devinit(mbi_addr)
    n ← get_size()
    highest ← 0

    FOR i ← 0 TO n-1 DO
        highest ← MAX(highest, get_mms(i) + get_mml(i))
    END FOR

    phys_nps ← highest / PAGESIZE
    set_nps(VM_USERHI_PI)
    pmm_init_freelists()

    // reset AT
    FOR i ← 0 TO get_nps()-1 DO
        at_set_allocated(i,0)
        at_set_perm(i,0)
        AT[i] ← {order:0, next:-1, prev:-1}
    END FOR

    // kernel reserved
    FOR i ← 0 TO VM_USERLO_PI-1 DO
        at_set_perm(i,1)
    END FOR

    // mark RAM / holes
    FOR i ← VM_USERLO_PI TO VM_USERHI_PI-1 DO
        phys ← i - VM_USERLO_PI

        IF phys ≥ phys_nps THEN
            at_set_perm(i,0)
```

```
            CONTINUE
        END IF

        is_ram ← FALSE
        FOR j ← 0 TO n-1 DO
            IF is_usable(j) AND
               get_mms(j) ≤ phys*PAGESIZE AND
               phys*PAGESIZE + PAGESIZE ≤ get_mms(j)+get_mml(j)
            THEN
                is_ram ← TRUE
                BREAK
            END IF
        END FOR

        IF is_ram THEN
            at_set_perm(i,2)
        ELSE
            at_set_perm(i,0)
        END IF
    END FOR

    // buddy blocks
    i ← VM_USERLO_PI
    WHILE i < VM_USERHI_PI DO
        IF AT[i].perm ≠ 2 OR AT[i].allocated ≠ 0 THEN
            i ← i + 1
            CONTINUE
        END IF

        FOR order ← MAX_ORDER-1 DOWNTO 0 DO
            size ← 2^order
            IF (i MOD size = 0) AND is_block_free_normal(i,order) THEN
                BREAK
            END IF
        END FOR

        IF order < 0 THEN
            i ← i + 1
        ELSE
            AT[i].order ← order
            at_list_add(order,i)
            i ← i + 2^order
        END IF
    END WHILE
END FUNCTION
```

Then in the **MATOp** layer, we added the physical layer page allocation and deallocation strategy.

```
FUNCTION palloc_order(order)
    IF order ≥ MAX_ORDER THEN return -1

    k ← order
    WHILE k < MAX_ORDER AND free_list[k] = empty DO
        k ← k + 1
    END WHILE

    IF k = MAX_ORDER THEN return -1
    p ← pop_free_list(k)

    WHILE k > order DO
        k ← k - 1
        buddy ← p + 2^k
        mark buddy free, order k
        add buddy to free_list[k]
    END WHILE

    size ← 2^order

    FOR i ← 0 TO size-1 DO
        AT[p+i].allocated ← 1
    END FOR

    AT[p].order ← order

    FOR i ← 1 TO size-1 DO
        AT[p+i].order ← 0
    END FOR

    AT[p].allocated ← 1

    return p

END FUNCTION
```

```
FUNCTION pfree_order(pindex)

    order ← AT[pindex].order

    WHILE order < MAX_ORDER-1 DO
        size ← 2^order
```

```
        clear allocated for block [pindex .. pindex+size-1]
        buddy ← pindex XOR size

        IF buddy out of range OR
           AT[buddy].order ≠ order OR
           buddy is allocated THEN
            BREAK
        END IF

        remove buddy from free_list[order]

        IF buddy < pindex THEN
            pindex ← buddy
        END IF

        order ← order + 1
        AT[pindex].order ← order

    END WHILE

    final_size ← 2^order
    clear allocated for [pindex .. pindex+final_size-1]

    add pindex to free_list[order]

END FUNCTION
```

In **MContainer** layer, we extended the container logic with page allocation and deallocation function calls

```
FUNCTION container_alloc_superpage(id)

    IF container_can_consume(id, PAGES_PER_SUPERPAGE) THEN

        pindex ← palloc_superpage()

        IF pindex ≠ 0 THEN
            CONTAINER[id].usage += PAGES_PER_SUPERPAGE
            RETURN pindex
        END IF
    END IF

    RETURN 0

END FUNCTION
```

```
FUNCTION container_free(id, page_index)

    order ← at_get_order(page_index)

    IF order = SUPERPAGE_ORDER THEN
        pfree_superpage(page_index)
        CONTAINER[id].usage -= PAGES_PER_SUPERPAGE
    ELSE
        pfree(page_index)
        IF CONTAINER[id].usage > 0 THEN
            CONTAINER[id].usage -= 1
        END IF
    END IF

END FUNCTION
```

As the **MPTIntro, MPTInit, MPTKern** and **MATOp** layer have very basic helper functions for the virtual memory layer, we haven't added much significant logic. Rather in the **MPTComm** layer, we added the superpage allocation and deallocation logic from the Virtual layer.

```
FUNCTION alloc_superpage(proc_index, vadr)

    pindex ← container_alloc_superpage(proc_index)
    IF pindex = 0 THEN return 0

    pde_index ← vadr >> 22
    pde_entry ← (pindex << 12) OR 0x87

    set_pde(proc_index, pde_index, pde_entry)

    RETURN pindex

END FUNCTION
```

```
FUNCTION free_ptbl(proc_index, vadr)

    pde ← get_pdir_entry_by_va(proc_index, vadr)

    IF pde NOT present THEN return

    IF pde is superpage THEN
```

```
        page_index ← pde >> 12
        remove_pde(proc_index, vadr)
        container_free(proc_index, page_index)
        RETURN
    END IF

    page_index ← pde >> 12
    remove_pde(proc_index, vadr)
    container_free(proc_index, page_index)

END FUNCTION
```

Then in the **MPTNew** layer, we added the space allocation logic, through what we can allocate arbitrary size of space, and according to the requirements we'll allocate superpages and normal pages for it.

```
FUNCTION alloc_space(proc_index, vaddr, size, perm)

    npages ← ceil(size / PAGESIZE)

    superpages_needed ← npages / 1024
    pages_needed ← npages % 1024

    IF NOT container_can_consume(proc_index, npages) THEN
        return MagicNumber
    END IF

    current_vaddr ← vaddr
    allocated_superpages ← 0

    FOR i ← 0 TO superpages_needed - 1 DO
        sp ← container_alloc_superpage(proc_index)

        IF sp = 0 THEN goto rollback

        IF map_page(proc_index, current_vaddr, sp, perm OR PTE_PS) = MagicNumber
THEN
            container_free(proc_index, sp)
            goto rollback
        END IF

        current_vaddr ← current_vaddr + SUPERPAGE_SIZE
        allocated_superpages++
    END FOR
```

```
    remaining ← npages - allocated_superpages * 1024

    FOR i ← 0 TO remaining - 1 DO
        p ← container_alloc(proc_index)

        IF p = 0 THEN goto rollback

        IF map_page(proc_index, current_vaddr, p, perm) = MagicNumber THEN
            container_free(proc_index, p)
            goto rollback
        END IF

        current_vaddr ← current_vaddr + PAGESIZE
    END FOR

    return vaddr


rollback:

    current_vaddr ← vaddr

    FOR i ← 0 TO allocated_superpages - 1 DO
        pde ← get_pdir_entry_by_va(proc_index, current_vaddr)

        IF pde present THEN
            p ← pde >> 12
            unmap_page(proc_index, current_vaddr)
            container_free(proc_index, p)
        END IF

        current_vaddr ← current_vaddr + SUPERPAGE_SIZE
    END FOR

    allocated_pages ← (npages - allocated_superpages * 1024)

    FOR i ← 0 TO allocated_pages - 1 DO
        pte ← get_ptbl_entry_by_va(proc_index, current_vaddr)

        IF pte present THEN
            p ← pte >> 12
            unmap_page(proc_index, current_vaddr)
            container_free(proc_index, p)
        END IF

        current_vaddr ← current_vaddr + PAGESIZE
    END FOR
```

```
    return MagicNumber

END FUNCTION
```

In the **MPTHeap** layer, we added the final brk() system call which will trigger the heap operation for a process.

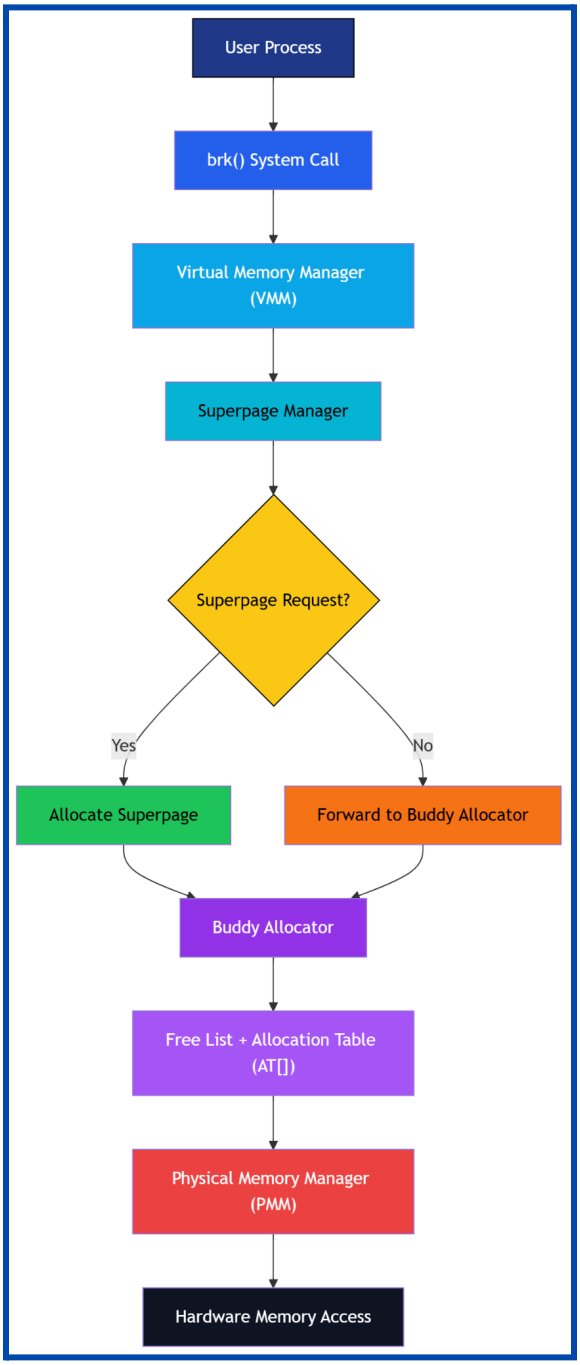
```
FUNCTION sys_brk_set(proc_index, new_brk)

    old_brk ← heap_break[proc_index]

    IF new_brk < VM_USERLO OR new_brk ≥ VM_USERHI THEN
        return old_brk
    END IF

    IF new_brk > old_brk THEN
        addr ← old_brk

        WHILE addr < new_brk DO
            IF ptbl_entry(addr) ≠ 0 THEN
                addr ← addr + PAGESIZE
                CONTINUE
            END IF

            p ← container_alloc(proc_index)
            IF p = 0 THEN return old_brk

            set_ptbl_entry(addr, p, PTE_P | PTE_W | PTE_U)
            addr ← addr + PAGESIZE
        END WHILE
    END IF

    IF new_brk < old_brk THEN
        addr ← new_brk

        WHILE addr < old_brk DO
            pte ← ptbl_entry(addr)
            IF pte ≠ 0 THEN
                p ← pte >> 12
                remove_ptbl_entry(addr)
                container_free(proc_index, p)
            END IF

            addr ← addr + PAGESIZE
        END WHILE
```

```
    END IF

    heap_break[proc_index] ← new_brk

    return new_brk

END FUNCTION
```

And finally we added monitoring logics through the TLBMonitor layer, which will check the hit and miss ratio.

```
FUNCTION lookup_tlb(proc_id, vaddr)

    FOR i ← 0 TO TLB_SUPERPAGE_ENTRIES-1 DO

        IF super_tlb[i] valid AND super_tlb[i].proc_id = proc_id THEN

            IF vaddr in super_tlb[i] range THEN
                update stats
                return translated superpage address
            END IF

        END IF

    END FOR

    FOR i ← 0 TO TLB_ENTRIES-1 DO

        IF regular_tlb[i] valid AND regular_tlb[i].proc_id = proc_id THEN

            IF vaddr in regular_tlb[i] range THEN
                update stats
                return translated page address
            END IF

        END IF

    END FOR

    stats.misses++

    return 0

END FUNCTION
```

# The System Architecture




The system is designed using a layered memory management architecture that efficiently handles dynamic memory allocation. A user process initiates a memory request through the brk() system call, which is handled by the Virtual Memory Manager (VMM). The VMM then evaluates the request and decides whether to allocate memory as a superpage (for large contiguous memory) or proceed with standard allocation.

The request is forwarded to the buddy allocator. The buddy allocator manages physical memory using a 2^order system, enabling efficient allocation through block splitting and deallocation through merging. It relies on a free list and Allocation Table (AT[]) to track memory blocks and maintain fast operations.

Finally, the Physical Memory Manager (PMM) maps the allocated memory to actual hardware, ensuring correct memory access.

# Tools and Technologies

**Operating System Framework**

- **mCertiKOS** – A modular and educational operating system framework used as the base system, providing core kernel functionalities and memory management infrastructure.

**Programming Language**

- **C** – Default used for implementing mCertikOS every functionalities such as low-level memory management, including the Physical Memory Manager (PMM), buddy allocator, and system call handling.

**Platform / Architecture**

- **x86 CPU Architecture** – The system utilizes features such as paging, page directories, page tables, and Page Size Extension (PSE) for superpages.

**Development Tools**

- **VMware** – Used to create a virtualized environment for running and testing the **mCertikOS** operating system safely without affecting the host machine.
- **VS Code (Visual Studio Code)** – Used as the primary code editor for writing, debugging, and managing the project source code inside the kernel.
- **GCC Compiler** – Used to compile C programs and build the OS kernel.
- **Makefile** – Used to automate the build process.
- **QEMU** – Used to emulate hardware and run the OS kernel for testing and debugging inside the kernel environment.

**Algorithms/Data Structures Used**

- **Allocation Table** - This stores metadata for each page : Allocation status, Order, Permissions.
- **Free list array** - This implements power-of-two memory allocation (2^order) and uses a free list data structure for each order. This supports block splitting and merging.
- **Buddy Allocation Algorithm** - Allocates memory in blocks of size 2^order using the free list architecture. This ensures fast allocation and deallocation
- **LRU (Least Recently Used)** - Related to TLB logic and caching strategies to find the hit and miss ratio.

**Core Operating System Concepts Applied**

- **Main Memory Management** – Handling allocation and deallocation of physical memory
- **Paging & Virtual Memory** – Mapping virtual addresses to physical addresses
- **Page Tables & Page Directories** – Used for address translation
- **Two-Level Paging** – Hierarchical structure for efficient memory mapping
- **Memory Mapping** – Linking virtual memory to physical memory
- **System Calls (brk)** – Interface between user processes and kernel memory management
- **Translation Lookaside Buffer (TLB)** – Cache for faster address translation
- **Page Size Extension (PSE)** – Enables superpage (large page) support
- **Fragmentation Management** – Reducing external fragmentation using buddy system
- **Contiguous Memory Allocation** – Allocating physically adjacent memory blocks

# Problems Encountered & Solutions

**Allocation Failure for Large Requests**

- **Issue:** No contiguous blocks available for a large space request. This will cause too many page tables overhead.
- **Solution:** Introduced superpage allocation with fallback mechanism. If we cannot allocate superpages, we will try to allocate contagious 4kb pages. In the worst case, we'll allocate scattered pages.

**Heap Expansion Failure**

- **Issue:** Partial allocation failure caused inconsistency; even for a user process.
- **Solution:** Implemented rollback mechanism in brk() system call for growth mechanism.

**Efficient Metadata Management**

- **Issue:** Tracking pages was complex with the original Allocation Table's members.
- **Solution:** Introduced additional members in the Allocation Table (AT[]) struct like order.

**Kernel Panic During Testing**

- **Issue:** The system encountered kernel panic due to improper sequencing of memory operations and test case execution, leading to invalid memory access and system crashes.
- **Solution:** Reordered and refined the test case calling mechanisms to ensure proper initialization and safe execution order, preventing invalid states and stabilizing the system.

# Setup Instructions

### Required Tools, and Dependencies

- GCC Compiler
- Linux environment / VM
- QEMU (for OS simulation)
- Makefile-based build system

### Clone Repository

```
git clone <https://github.com/ZAsabiha/Advanced_Memory_Allocation_Os.git>
cd project-folder
```

### Remove Temporary Files While Building Phase

```
make clean
```

### Enable the Test Cases

```
make TEST=1
```

### Test Cases We've Written:

- Basic Allocation
- Free list validity checking
- Alignment validation checking
- Basic buddy Splitting and merging
- Request large memory
- Confirm superpage mapping
- Rollback of unsuccessful allocations
- User space  freeing and merging
- Heap growth and shrinking (brk)
- Page accessing through virtual address
- TLB monitoring ratio of hit and miss

# Team Members & Contributions

| Member Name | Student ID | Contribution |
|---|---|---|
| Israt Risha Ivey | 220042103 | Page accessing though virtual address, superpage allocation logic in PMM layer |
| Ramisa Anan Rahman | 220042105 | Heap growth and shrinking of a user process, brk() system call |
| Ridika Naznin | 220042115 | Page alignment, user process space allocation through virtual address |
| Zannatul Adon Sabiha | 220042123 | Buddy allocation, free list, allocation table, page allocating and freeing in PMM layer, superpage allocation and freeing in VMM layer |
| Jarin Subha Sneha | 220042161 | Superpage allocation logic in PMM layer, TLB monitoring with hit,miss including eviction logic |

# Limitation & Future Scope

Superpage allocation depends on a strict **4MB** size, making it less flexible under heavy fragmentation. The system also does not support advanced page replacement strategies or dynamic optimization techniques, and debugging at the kernel level remains complex due to limited visibility and higher risk of system crashes such as kernel panic.

The system can be further improved by introducing more flexible allocation strategies to reduce internal fragmentation, such as hybrid allocators. Support for variable-sized superpages and smarter allocation policies can enhance performance under diverse workloads. Additionally, integrating advanced page replacement algorithms and memory optimization techniques can improve efficiency. Further enhancements may also include extending support for multi-core environments and optimizing TLB performance.

In conclusion, this project demonstrates how memory is structured and managed using a clear address segmentation approach. By separating the address into PDE and offset parts, it becomes easier to understand how large memory spaces are efficiently mapped. Overall, it provides a simplified but effective model of virtual memory translation and system-level memory organization.